

## Full-Stack Web Development Bootcamp

### 1. Bootcamp Details

- **Target Age Group:** Ages 13 - 18
- **Duration:** 8 Weeks
- **Schedule:**
  - **Live Sessions:** 3 per week
  - **Duration per Session:** 1 hour
  - **Total Sessions:** 24
  - **Sessions on:** Monday, Wednesday, Friday
  - **Session held at:** 6:00 PM PST
- **Format:**
  - **Interactive Coding Exercises:** Real-time tasks for concept application.
  - **Q&A Segments:** Time for questions and instructor feedback.
- **Target Audience:** Beginners with a basic understanding of computers, eager to learn web development.
- **Pricing:** \$599 per participant
- **Tools Required:**
  - **Google Meet, Slack, GitHub, Visual Studio Code**
- **Scholarships:**
  - **Number Available:** 5 per bootcamp
  - **Types:** Partial and full scholarships.



### **Benefits:**

- **Verifiable Dyne Research Certificate:** Receive a certificate that can be verified for authenticity online.
  - **Letters of Recommendation:** Top 5 students will receive personalized letters of recommendation.
  - **Personalized Completion Certificate and LOR (if applicable):** Completion certificates and letters of recommendation will be delivered personally.
  - **Possible Internship with Dyne Research:** Opportunity to intern with Dyne Research, subject to availability and performance.
- 

## **2. Curriculum Outline**

### **Week 1: Introduction and Basics**

#### **Day 1: Introduction to Web Development**

- **Objective:** Understand the fundamentals of web development.
- **Activities:**
  - **Lecture:** Overview of front-end vs. back-end development.
  - **Demo:** Introduction to HTML and CSS.
  - **Setup:** Guide participants through setting up their development environment.
- **Assignments:**
  - Install necessary tools (VS Code, Git, etc.).
  - Complete a basic tutorial on HTML and CSS.

## Day 2: HTML Deep Dive

- **Objective:** Master HTML basics.
- **Activities:**
  - **Lecture:** HTML elements, attributes, and structure.
  - **Interactive Exercise:** Build a basic webpage including headers, paragraphs, links, and images.
  - **Q&A:** Address any questions about HTML.
- **Assignments:**
  - Create a personal webpage using various HTML elements.
  - Submit the webpage to GitHub.

## Day 3: CSS Basics

- **Objective:** Learn fundamental CSS styling techniques.
- **Activities:**
  - **Lecture:** CSS syntax, selectors, box model, positioning, and layout techniques.
  - **Interactive Exercise:** Apply CSS styles to the HTML page created previously.
  - **Q&A:** Troubleshoot CSS issues and clarify concepts.
- **Assignments:**
  - Enhance the personal webpage with custom styles.
  - Experiment with layout techniques and submit updates.

---

## Week 2: Front-End Framework

### Day 1: Introduction to React



- **Objective:** Learn the basics of React.
- **Activities:**
  - **Lecture:** Introduction to React, components, props, and state.
  - **Interactive Exercise:** Create and manage simple React components.
  - **Q&A:** Address any initial challenges with React.
- **Assignments:**
  - Build a small React application with components and props.
  - Submit the React project to GitHub.

## Day 2: Advanced React Concepts

- **Objective:** Dive deeper into React features.
- **Activities:**
  - **Lecture:** React Hooks, context API.
  - **Interactive Exercise:** Use Hooks and context API to manage state in a React application.
  - **Q&A:** Troubleshoot issues with advanced React concepts.
- **Assignments:**
  - Extend the React application with Hooks and context API.
  - Submit updated code to GitHub.

## Day 3: State Management and Routing

- **Objective:** Implement state management and routing in React.
- **Activities:**
  - **Lecture:** Using React Router for navigation and introduction to state management libraries (e.g., Redux).

- **Interactive Exercise:** Add routing and state management to the React application.
  - **Q&A:** Address questions about state management and routing.
  - **Assignments:**
    - Implement React Router and a state management solution in the project.
    - Submit the updated React application.
- 

## Week 3: Python 3 Basics for Django

### Day 1: Introduction to Python 3

- **Objective:** Familiarize with the basics of Python 3 syntax and data types.
- **Activities:**
  - **Lecture: Python 3 Basics:**
    - Introduction to Python and its features.
    - Basic syntax, variables, and data types (int, float, str, bool).
    - Input and output operations.
  - **Interactive Exercise: Basic Python Script:**
    - Write a script to perform basic arithmetic operations.
    - Create a simple program that takes user input and prints a message.
  - **Q&A: Python Basics:**
    - Address questions related to Python syntax and data types.
- **Assignments:**
  - **Task 1:** Write a Python script to calculate the area of a circle given its radius.



- **Task 2:** Create a script that takes a user's name as input and prints a personalized greeting.

## Day 2: Control Flow and Functions

- **Objective:** Understand control flow statements and functions in Python.
- **Activities:**
  - **Lecture: Control Flow and Functions:**
    - Conditional statements (if, elif, else).
    - Loops (for, while).
    - Functions: definition, parameters, return values.
  - **Interactive Exercise: Control Flow and Functions:**
    - Write a program using conditional statements to determine if a number is positive, negative, or zero.
    - Create a function that calculates the factorial of a number.
  - **Q&A: Control Flow and Functions:**
    - Troubleshoot common issues with control flow and functions.
- **Assignments:**
  - **Task 1:** Implement a function to check if a given string is a palindrome.
  - **Task 2:** Write a program that prints the Fibonacci series up to a specified number.

## Day 3: Data Structures and Modules

- **Objective:** Learn about basic data structures and modules in Python.
- **Activities:**
  - **Lecture: Data Structures and Modules:**

- Lists, tuples, sets, and dictionaries.
  - Importing and using modules.
  - Introduction to standard libraries useful for Django.
  - **Interactive Exercise: Data Structures:**
    - Create and manipulate lists, tuples, sets, and dictionaries.
    - Write a program to read a list of numbers and sort them in ascending order.
  - **Q&A: Data Structures and Modules:**
    - Address questions related to data structures and module usage.
  - **Assignments:**
    - **Task 1:** Write a Python script to count the frequency of each word in a given text file.
    - **Task 2:** Create a module with utility functions and demonstrate its usage in a script.
- 

## Week 4: Back-End Development

### Day 1: Introduction to Django

- **Objective:** Set up a Django project and understand its basic components.
- **Activities:**
  - **Lecture: Setting Up Django:**
    - Installation of Django and project setup using django-admin.
    - Overview of Django's architecture, including models, views, and templates.

- **Interactive Exercise: Create a Simple Django Project:**
  - Set up a new Django project.
  - Create a basic Django app within the project.
  - Develop a simple view to display “Hello, World!” and a basic template.
- **Q&A: Django Setup Issues:**
  - Troubleshoot common issues with Django setup.
  - Answer questions about the project structure and configuration.
- **Assignments:**
  - **Task 1:** Set up a basic Django project and create a simple app.
  - **Task 2:** Push the project to GitHub and include basic instructions for setup.

## Day 2: API Request Basics

- **Objective:** Build and handle APIs with Django.
- **Activities:**
  - **Lecture: Introduction to RESTful APIs:**
    - Basics of RESTful APIs and their role in web development.
    - Overview of Django-Rest-Framework for building APIs.
  - **Interactive Exercise: Create a Simple API:**
    - Set up Django-Rest-Framework in the existing Django project.
    - Develop a basic API endpoint to perform CRUD operations on a simple model.
    - Test the API using tools like Postman.
  - **Q&A: API Development Issues:**

- Address any challenges with API creation and testing.
- Provide guidance on handling API requests and responses.
- **Assignments:**
  - **Task 1:** Develop a RESTful API endpoint in the Django project.
  - **Task 2:** Submit the updated project to GitHub, including API documentation.

### Day 3: Working with Databases

- **Objective:** Learn to manage databases in Django.
- **Activities:**
  - **Lecture: Introduction to Django Models and Databases:**
    - Overview of Django models and their role in ORM (Object-Relational Mapping).
    - Explanation of SQLite and database migrations.
  - **Interactive Exercise: Create and Manage Database Models:**
    - Define a Django model and set up database tables using migrations.
    - Perform CRUD operations on the database via Django's admin interface and Django shell.
  - **Q&A: Database Integration Issues:**
    - Troubleshoot issues related to model creation, migrations, and database operations.
    - Discuss best practices for database management in Django.
- **Assignments:**

- **Task 1:** Add database models to the Django project and perform migrations.
  - **Task 2:** Submit an updated Django project to GitHub, demonstrating database functionality.
- 

## Week 5: Full-Stack Integration

### Day 1: Connecting Front-End to Back-End

- **Objective:** Integrate React with Django.
- **Activities:**
  - **Lecture:** Making API requests from React to Django, handling data flow between client and server.
  - **Interactive Exercise:** Connect the React front-end with the Django back-end.
  - **Q&A:** Address issues with front-end and back-end integration.
- **Assignments:**
  - Complete the integration of React and Django for a functional application.
  - Submit the full-stack project to GitHub.

### Day 2: Authentication and Authorization

- **Objective:** Implement authentication and authorization.
- **Activities:**
  - **Lecture:** Implementing user authentication (e.g., JWT), managing user sessions and permissions.

- **Interactive Exercise:** Add authentication to the full-stack application.
- **Q&A:** Resolve any challenges with authentication.
- **Assignments:**
  - Implement user authentication and authorization in the project.
  - Submit an updated project with authentication features.

### Day 3: Advanced Full-Stack Topics

- **Objective:** Handle advanced full-stack development topics.
  - **Activities:**
    - **Lecture:** Error handling, validation, optimizing performance and security.
    - **Interactive Exercise:** Improve error handling and performance in the application.
    - **Q&A:** Discuss advanced topics and troubleshoot issues.
  - **Assignments:**
    - Optimize the full-stack project for performance and security.
    - Submit the final full-stack project.
- 

## Week 6: Deployment

### Day 1: Introduction to Deployment

- **Objective:** Learn about deployment platforms.
- **Activities:**
  - **Lecture:** Overview of deployment platforms (e.g., Heroku, Netlify).
  - **Demo:** Walkthrough of the deployment process.

- **Q&A:** Answer questions about deployment.
- **Assignments:**
  - Prepare the project for deployment.
  - Document deployment steps.

## Day 2: Deploying Front-End Applications

- **Objective:** Deploy React applications.
- **Activities:**
  - **Lecture:** Deploying React apps to Netlify.
  - **Interactive Exercise:** Deploy a React application to Netlify.
  - **Q&A:** Resolve issues with deployment.
- **Assignments:**
  - Deploy the React front-end application and ensure it's live.
  - Submit deployment documentation.

## Day 3: Deploying Full-Stack Applications

- **Objective:** Deploy full-stack applications.
- **Activities:**
  - **Lecture:** Deploying Django applications to Heroku, handling environment variables and configuration.
  - **Interactive Exercise:** Deploy a Django application to Heroku.
  - **Q&A:** Troubleshoot deployment issues.
- **Assignments:**
  - Deploy the full-stack application to Heroku.
  - Submit final deployment and project documentation.

## Week 7: Project Work

### Day 1: Project Planning

- **Objective:** Plan and prepare for the final project.
- **Activities:**
  - **Lecture:** Overview of final project requirements.
  - **Interactive Workshop:** Brainstorming and project planning.
  - **Q&A:** Discuss project ideas and plans.
- **Assignments:**
  - Develop a project plan with objectives and timelines.
  - Submit the project plan for review.

### Day 2: Project Development

- **Objective:** Develop the final project.
- **Activities:**
  - **Workshop:** Coding and building the final project.
  - **Live Coding:** Troubleshoot and refine the project.
  - **Q&A:** Provide support and guidance during development.
- **Assignments:**
  - Work on coding and implementing the final project.
  - Prepare for a project presentation.

### Day 3: Project Review and Feedback

- **Objective:** Review and receive feedback on final projects.

- **Activities:**
    - **Presentation:** Present final projects to peers and instructors.
    - **Feedback Session:** Peer review and instructor feedback.
  - **Assignments:**
    - Revise project based on feedback.
    - Submit the final version of the project.
- 

## Week 8: Career Preparation and Wrap-Up

### Day 1: Resume and Portfolio Building

- **Objective:** Prepare for the job market.
- **Activities:**
  - **Workshop:** Creating a developer portfolio and resume.
  - **Lecture:** Resume tips, interview preparation.
  - **Mentorship:** Meet with mentors for career advice.
- **Assignments:**
  - Build or update your portfolio and resume.
  - Submit for review.

### Day 2: Job Search Strategies

- **Objective:** Develop job search skills.
- **Activities:**
  - **Lecture:** Networking and job search techniques.
  - **Workshop:** Navigating the tech job market.

- **Assignments:**
  - Create a job search plan.
  - Network with industry professionals.

### Day 3: Final Q&A and Course Wrap-Up

- **Objective:** Wrap up the bootcamp.
  - **Activities:**
    - **Q&A Session:** Address any final questions.
    - **Review:** Provide additional resources and next steps.
    - **Closure:** Course feedback and evaluation.
  - **Assignments:**
    - Complete any outstanding tasks or revisions.
    - Provide feedback on the bootcamp experience.
- 

## 3. Interactive Elements

- **Live Coding Exercises:** Real-time coding challenges during sessions.
- **Breakout Sessions:** Small group discussions and coding labs.
- **Q&A:** Dedicated time for questions and troubleshooting.
- **Office Hours:** Weekly one-on-one support sessions.